

logic, custom headers, or handing a request off to something other than the built-in downloader. Spider middleware sits between the engine and your spider, processing the responses that come in and the items and requests that go out. You rarely write these from scratch, but they are the seam that lets Scrapy integrate cleanly with other tools, which is exactly what we do next.

Composing the two: Scrapy for scale, Zyte API for the walls

Scrapy is a superb crawler, but on its own it uses a plain HTTP downloader, so it hits the same three walls as any other client. The natural move is to keep Scrapy as the engine and route its actual fetching through the Zyte API, which is what the *scrapy-zyte-api* package does. Zyte's own convention for wiring up Scrapy plugins is the add-on system, so setup is a couple of lines in your project's *settings.py* rather than a scattering of middleware entries.

```
# settings.py
ADDONS = {
    "scrapy_poet.Addon": 300,
    "scrapy_zyte_api.Addon": 500,
}
```

You supply your key through the `ZYTE_API_KEY` environment variable or the same setting, and the add-on does the rest. It registers the download handlers, the downloader and spider middlewares, and the request fingerprinter, switches Scrapy to the asyncio reactor, and turns on transparent mode so that every request your spiders make is routed through the Zyte API automatically. Unblocking and, when you ask for it, browser rendering now apply to your whole crawl without touching a single spider.

The *scrapy_poet.Addon* line is what makes the next step work, and it is easy to miss. With *scrapy-poet* in the mix, you can declare what data you want as a type annotation on your callback and let it be injected, extraction and all. A

complete product spider looks like this:

```
import scrapy
from zyte_common_items import Product
class BooksSpider(scrapy.Spider):
    name = "books"
    start_urls = [
        "https://books.toscrape.com/
catalogue/a-light-in-the-attic_1000/
index.html"
    ]
    def parse(self, response, product:
Product):
        yield product
```

That *product: Product* argument is the whole trick. *scrapy-poet* sees the annotation, asks the Zyte API to extract a product from the page, and hands your callback a fully populated *Product* object. Running this spider yielded one item with the name *A Light in the Attic*, a price of 51.77 in GBP, InStock availability, and the complete *Home > Books > Poetry* breadcrumb trail, and the crawl stats confirmed the request had gone out with the product set. You now have Scrapy's crawling, concurrency, and pipelines on the outside, and structured extraction with no selectors on the inside. If I had left out the *scrapy-poet* add-on, that same spider would fail with a missing-argument error, because nothing would be there to inject the product, which is why both add-on lines earn their place in the settings.

Zyte API provides an array of other functionality as well, like changing your request geolocation to get localised content and taking screenshots of a page from API call. These are out of scope of this article but feel free to refer the official documentation at <https://docs.zyte.com/>.

When to use what

The point of separating the crawl from the hard parts is that you can spend effort and money only where a wall actually stands. It is easy to reach for a paid API by reflex, so here is the decision I made (Figure 4).

If the site is static, cooperative, and you need a page or two on a one-off basis, do not overthink it. *requests* or *httpx* with BeautifulSoup are the right tools – they are free, and adding anything more is wasteful. Open source gets you a long way, and a lot of useful scraping never needs more than this.

If you are crawling a whole site, running the job on a schedule, or working across thousands of pages, that is when Scrapy pays for itself. The concurrency, retries, throttling, and pipelines you would otherwise reinvent are exactly what it gives you, and it is still entirely open source.

From there, add capability only against the wall you actually hit. If you are getting blocked, turn on unblocking and let the API manage proxies and fingerprints instead of running that fleet yourself. If the content is rendered client-side, ask for browser rendering rather than standing up and babysitting your own headless browsers. And if you are tired of selectors breaking, or you need the same product, article, or job-posting fields across many different sites, automatic extraction is where the API changes the economics of the whole project. The rule I follow is to crawl with open source and reach for the API at the wall, once I have actually hit one.

Bonus: Let an AI coding agent write the scraper for you

One more shift is changing how the code above gets written in the first place. Zyte publishes a plugin for Claude Code, the terminal-based AI coding agent, distributed as a set of agent skills rather than as an MCP server. You install it once:

```
claude plugin marketplace add zytedata/
claude-skills
claude plugin install zyte-web-data@
zyte-ai
```

Then you describe the site in plain language, something as short as 'Scrape books.toscrape.com', and the agent